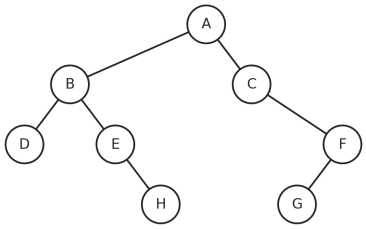
1. (4 Punkte) Beantworte die folgenden Fragen zum abgebildeten Baum.



   (a) (1 Punkt) Welcher Knoten ist die Wurzel?

   (b) (1 Punkt) Welche Knoten sind Blätter?

   (c) (1 Punkt) Wie heißen der Elternknoten von E und die Kinder von C?

   (d) (1 Punkt) Auf welcher Ebene liegt der Knoten G?

Lösung: a) A
b) D, H, G
c) Der Elternknoten von E ist B. C hat nur ein Kind, nämlich F.
d) Ebene 3

2. (3 Punkte) Die Wurzel eines binären Baums liegt auf Ebene 0.

   (a) (1 Punkt) Wie viele Knoten kann ein binärer Baum auf Ebene 3 höchstens haben?

   (b) (1 Punkt) Wie viele Knoten kann er auf Ebene k höchstens haben?

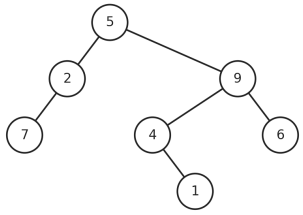
   (c) (1 Punkt) Wie viele Knoten hat ein Baum, bei dem alle Ebenen von 0 bis 4 vollständig besetzt sind?

Lösung: a) 8, denn jeder Knoten hat höchstens zwei Kinder. Die Anzahl verdoppelt sich mit jeder Ebene.
b) 2^k
c) $1 + 2 + 4 + 8 + 16 = 31$

3. (3 Punkte) Die Funktion `baumString` liefert für den Baum b die folgende Ausgabe. Zeichne den Baum.

```
..6
.9
...1
..4
5
.2
..7
```

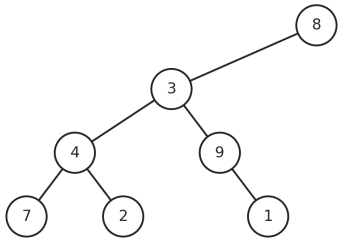
Lösung:



4. (3 Punkte) Zeichne den Baum, auf den die Variable w verweist.

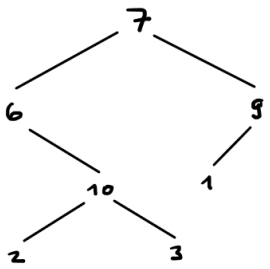
```
x = Node(4, Node(7), Node(2))
y = Node(9, None, Node(1))
z = Node(3, x, y)
w = Node(8, z, None)
```

Lösung:



5. (5 Punkte) a. Schreibe eine Klasse `Node`, die einen Knoten eines binären Baums repräsentiert.

   b. Schreibe eine Funktion `baum`, die den abgebildeten Baum zurückgibt. Nutze dazu deine Node-Klasse. Verschachtele die Node-Aufrufe aus Lesbarkeitsgründen höchstens einmal und gib den temporären Teilbäumen sprechende Namen.



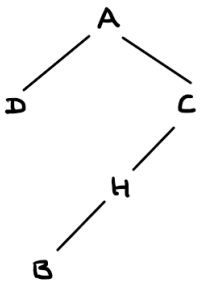
Lösung:

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def baum():
    b10 = Node(10,Node(2),Node(3))
    b6 = Node(6,None,b10)
    b9 = Node(9,Node(1),None)
    b7 = Node(7,b6,b9)
    return b7
```

6. (5 Punkte) a. Schreibe eine Klasse `Node`, die einen Knoten eines binären Baums repräsentiert.

   b. Schreibe eine Funktion `baum`, die den abgebildeten Baum zurückgibt. Nutze dazu deine Node-Klasse. Verschachtele die Node-Aufrufe aus Lesbarkeitsgründen höchstens einmal und gib den temporären Teilbäumen sprechende Namen.



Lösung:

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def baum():
    h = Node('H',Node('B'),None)
    c = Node('C',h,None)
    a = Node('A',Node('D'),c)
    return a
```

7. (3 Punkte) Gegeben ist das folgende Programm.

```
def f(node):
    if node is None:
        return 0
    return node.val + f(node.left)

baum = Node(2, Node(5, Node(1), Node(9)), Node(4))
print(f(baum))
```

   (a) (2 Punkte) Welche Ausgabe erscheint auf der Konsole?

   (b) (1 Punkt) Beschreibe in einem Satz, was die Funktion f berechnet.

Lösung: a) 8
b) die Summe der Werte auf dem Weg, der von der Wurzel aus immer nach links führt

8. (3 Punkte) Führe die Funktion von Hand aus. Welche Ausgabe erscheint auf der Konsole?

```
def g(node):
    if node is None:
        return
    g(node.left)
    g(node.right)
    print(node.val, end=' ')

baum = Node(1, Node(2, None, Node(3)), Node(4, Node(5), Node(6)))
g(baum)
```

Lösung: `3 2 5 6 4 1`

9. (3 Punkte) Die Funktion `summe(node)` gibt die Summe aller Werte im Baum mit der Wurzel node zurück. Für einen leeren Baum ist das Ergebnis 0. Implementiere die Funktion rekursiv.

Lösung:

```
def summe(node):
    if node is None:
        return 0
    return node.val + summe(node.left) + summe(node.right)
```

10. (3 Punkte) Die Funktion `enthaelt(node, x)` gibt True zurück, wenn der Wert x im Baum mit der Wurzel node vorkommt, sonst False. Implementiere die Funktion rekursiv.

Lösung:

```
def enthaelt(node, x):
    if node is None:
        return False
    if node.val == x:
        return True
    return enthaelt(node.left, x) or enthaelt(node.right, x)
```

11. (4 Punkte) Die Funktion `blaetter(node)` gibt die Anzahl der Blätter im Baum mit der Wurzel node zurück. Implementiere die Funktion rekursiv.

Lösung:

```
def blaetter(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return blaetter(node.left) + blaetter(node.right)
```

12. (3 Punkte) Die Funktion `anzahl` soll die Anzahl der Knoten eines Baums zurückgeben.

```
def anzahl(node):
    return 1 + anzahl(node.left) + anzahl(node.right)

print(anzahl(Node(5)))
```

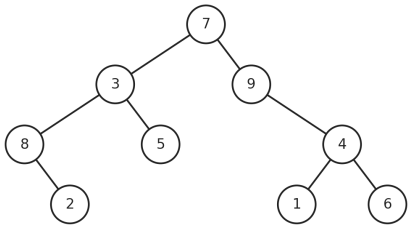
(a) (2 Punkte) Was passiert beim Aufruf? Begründe.

(b) (1 Punkt) Korrigiere die Funktion.

Lösung: a) Es gibt einen AttributeError. Der Basisfall fehlt: `anzahl` wird mit dem linken Kind aufgerufen, das ist None, und None hat kein Attribut left.
b)

```
def anzahl(node):
    if node is None:
        return 0
    return 1 + anzahl(node.left) + anzahl(node.right)
```

13. (4 Punkte) Gegeben ist der abgebildete Baum.



(a) (2 Punkte) In welcher Reihenfolge besucht die Tiefensuche die Knoten? (erst der Knoten, dann der linke, dann der rechte Teilbaum)

(b) (2 Punkte) In welcher Reihenfolge besucht die Breitensuche die Knoten?

Lösung: a) `7 3 8 2 5 9 4 1 6`
b) `7 3 9 8 5 4 2 1 6`

14. (4 Punkte) Für den Baum aus der vorigen Aufgabe wird eine Tiefensuche mit einem Keller durchgeführt. Notiere zeilenweise die Ausgabe und den Inhalt des Kellers, nachdem das rechte Kind (falls vorhanden) nach der Ausgabe in den Keller getan wurde. Notiere den Keller horizontal mit dem top-Element auf der linken Seite.

Lösung:

Ausgabe	Keller
	7
7	9
3	5 9
8	2 5 9
2	5 9
5	9
9	4
4	6
1	6
6	–

15. (4 Punkte) Für denselben Baum wird eine Breitensuche durchgeführt. Notiere zeilenweise die Ausgabe und den Inhalt der Schlange, nachdem die Kinder (falls vorhanden) nach der Ausgabe in die Schlange eingefügt wurden. Notiere die Schlange horizontal mit dem Kopf der Schlange auf der linken Seite.

Lösung:

Ausgabe	Schlange
	7
7	3 9
3	9 8 5
9	8 5 4
8	5 4 2
5	4 2
4	2 1 6
2	1 6
1	6
6	–

16. (3 Punkte) Vergleiche Tiefensuche und Breitensuche.
    (a) (1 Punkt) Welche Datenstruktur nutzt die Tiefensuche, um sich die noch offenen Teilbäume zu merken, welche die Breitensuche?
    (b) (1 Punkt) Warum eignet sich für die Breitensuche keine Datenstruktur, die nach dem LIFO-Prinzip arbeitet?
    (c) (1 Punkt) Ein gesuchter Wert liegt nah an der Wurzel in einem sehr großen Baum. Welches Verfahren findet ihn in der Regel schneller? Begründe.

Lösung: a) Die Tiefensuche nutzt einen Keller, die Breitensuche eine Schlange.
b) Die Knoten sollen ebenenweise besucht werden. Wer zuerst eingereiht wurde, muss deshalb auch zuerst drankommen (FIFO).
c) Die Breitensuche, denn sie besucht die Knoten Ebene für Ebene. Die Tiefensuche läuft vielleicht erst weit in einen falschen Teilbaum hinein.

17. (4 Punkte) Erkläre in ein bis zwei Sätzen.
    (a) (1 Punkt) Was ist ein binärer Baum?
    (b) (1 Punkt) Was ist ein Blatt?
    (c) (1 Punkt) Was bedeutet es, einen Baum zu traversieren?
    (d) (1 Punkt) Warum ist der Baum eine rekursive Datenstruktur?

Lösung: a) Ein binärer Baum ist entweder leer oder besteht aus einem Knoten, dem zwei binäre Bäume zugeordnet sind.
b) Ein Blatt ist ein Knoten ohne Kinder.
c) Jeden seiner Knoten besuchen.
d) Die Teilbäume eines Baums sind selbst wieder Bäume. Die Klasse Node verweist auf sich selbst.